

Segmentação distribuída de geleiras com gRPC e eleição de líder em anel

Bruno Menezes Rodrigues Oliveira Vaz, Davi Cândido de Almeida
Gabriel Valedo Batista Silva, João Pedro Torres
Vitor Leite Setragni, Welbert Junio Afonso de Almeida

[1]Instituto de Ciências Exatas e Informática – PUC Minas
Campus Coração Eucarístico – Belo Horizonte, MG – Brasil
Computação Distribuída – Prof. Luigi D. C. Soares – 2026.2 – Entrega 1

Abstract. *This work presents a prototype for distributed glacier segmentation inference. A coordinator sends overlapping image tiles to peers through gRPC, stitches probabilities and computes global pixel metrics. In case of a failure, the Chang-Roberts ring election replaces the failed coordinator. Job replication and recovery are reserved for the second delivery. Scientific accuracy with trained weights and real glacier data remains to be evaluated.*

Resumo. *Este trabalho apresenta um protótipo para inferência distribuída de segmentação de geleiras. Um coordenador envia tiles sobrepostos por gRPC, reúne probabilidades e calcula métricas globais. Em caso de falha, a eleição de Chang-Roberts em anel substitui o coordenador. A qualidade com pesos treinados e imagens reais permanece pendente de avaliação.*

1. Problema e escopo

O projeto tem como objetivo analisar o degelo glacial por meio da identificação da área ocupada por geleiras em imagens de satélite. Para viabilizar esse processamento de forma distribuída, a proposta desta etapa consiste em dividir uma cena em unidades menores, denominadas tiles, e distribuí-las entre diferentes nós da aplicação. A comunicação entre os nós é realizada por meio de chamadas remotas de procedimento (RPC), utilizando gRPC, enquanto a coordenação da execução é gerenciada por um mecanismo de eleição de líder implementado na própria aplicação.

A segmentação das imagens utiliza uma rede neural convolucional U-Net, cuja implementação e treinamento estão documentados no notebook `UNet.ipynb`. O modelo recebe imagens com 13 bandas espectrais e realiza a classificação dos pixels em duas classes: fundo e geleira, desconsiderando pixels inválidos. Sua arquitetura combina um caminho de contração, responsável pela extração de características, e um caminho de expansão, que realiza a reconstrução da segmentação, conectados por conexões de passagem de características (skip connections) [Ronneberger et al. 2015]. O notebook realiza treinamento centralizad. Nesta etapa, a distribuição ocorre exclusivamente durante a inferência: os nós utilizam os mesmos pesos do modelo, sem sincronização de gradientes ou treinamento independente para posterior combinação das máscaras.

2. Arquitetura e mensagens

Cada peer executa o mesmo serviço Python e pode assumir os papéis de coordenador ou worker. O cliente identifica o líder atual por meio de consultas de estado e submete uma solicitação de

processamento contendo um identificador único, a cena a ser analisada, um gabarito opcional e os parâmetros de divisão. O coordenador é responsável por organizar e distribuir os tiles entre os workers, receber os resultados da inferência e reconstruir a máscara final da cena. Os nós podem processar tiles simultaneamente, respeitando o limite de uma inferência por vez em cada worker. O próprio coordenador também pode participar do processamento como worker.

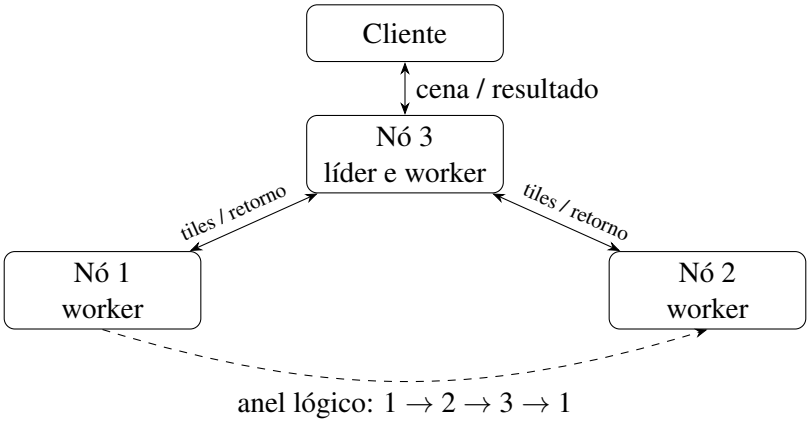


Figura 1. Fluxo de dados. Election e Coordinator percorrem o anel; Heartbeat consulta o estado dos peers. Os peers também participam da eleição em anel.

RPC	Comunicação	Conteúdo principal
SubmitJob	Cliente → líder	Submissão do identificador, cena, gabarito opcional e parâmetros de processamento.
GetJob	Cliente → líder	Consulta do estado do processamento e obtenção da máscara final.
InferTile	Líder → worker	Envio dos identificadores, posição, tensor, validade, hash do modelo e rodada; retorno das probabilidades.
Heartbeat	Peer → peer	Verificação de disponibilidade e consulta de informações sobre líder conhecido, rodada e hash do modelo.
Election	Peer → sucessor	Propagação da rodada de eleição e do candidato.
Coordinator	Peer → sucessor	Anúncio da rodada e do líder eleito.

Tabela 1. Resumo das chamadas RPC utilizadas no sistema.

Os membros do cluster e seus respectivos endereços são configurados previamente, e identificadores positivos e únicos determinam a ordem de participação no anel lógico. Caso o sucessor esperado não responda, o nó tenta estabelecer comunicação com o próximo membro configurado. Após uma eleição, o cliente passa a se comunicar com o novo líder. Cada peer mantém seus próprios arquivos localmente, não sendo necessário utilizar um sistema de armazenamento compartilhado entre nós.

2.1. Divisão, costura e métricas

Cada tile recebe um identificador único, derivado de sua posição na cena, de suas coordenadas (x, y) e de suas dimensões válidas. A configuração padrão utiliza tiles de 128×128 pixels, com sobreposição de 32 pixels entre unidades adjacentes, resultando em um passo (stride) de

96 pixels. Para garantir a cobertura de toda a cena, a posição do último tile em cada dimensão é ajustada à borda. Quando a cena possui dimensões menores que as do tile, aplica-se preenchimento (padding).

Na reconstrução da cena, o coordenador acumula as probabilidades de classificação e o número de contribuições recebidas para cada posição. Em seguida, calcula a média das probabilidades nos pixels cobertos e aplica o limiar de 0,5 para gerar a máscara binária final. Resultados associados a identificadores já processados são ignorados, evitando a contabilização duplicada de tiles em caso de reenvio. A média nas regiões de sobreposição permite combinar as predições de diferentes tiles, reduzindo possíveis divergências entre suas bordas. Entretanto, o impacto dessa estratégia na qualidade da segmentação ainda precisa ser avaliado.

3. Implementação dos requisitos

3.1. RPC e integração com a U-Net

O contrato Protobuf gera os stubs Python; o serviço e o cliente usam `grpc.aio`. A aplicação define prazos para as chamadas e repete tiles em outro worker quando há erro de comunicação. A definição de serviços e geração de stubs seguem a documentação do gRPC [gRPC 2026a]; prazos explícitos limitam a espera por respostas [gRPC 2026b].

As matrizes são transmitidas como bytes NPY, sem pickle. A aplicação valida dimensões, tipos, rótulos e tamanho antes de processá-las. Cada mensagem tem limite de 64 MiB e cada array de 48 MiB; o protótipo usa RPCs unários. O paralelismo é limitado por uma fila por worker e uma janela de tarefas, reduzindo o acúmulo de tiles na memória.

As classes da U-Net foram extraídas do notebook, mantendo nomes e formatos das camadas. O adaptador carrega a chave `modelo` do checkpoint, usa `eval()` e `inference_mode()` e obtém a probabilidade da classe geleira por softmax. A média e o desvio são os calculados no treino, nas mesmas 13 bandas. O hash dos pesos, da normalização e da ordem de bandas impede misturar versões de modelo no mesmo job. O dispositivo pode ser CPU ou GPU; a verificação executada nesta sessão foi em CPU.

As métricas são calculadas na máscara final, contando cada pixel válido uma única vez. Para verdadeiros positivos TP, falsos positivos FP e falsos negativos FN:

$$\text{IoU} = \frac{TP}{TP + FP + FN}, \qquad \text{Dice} = \frac{2TP}{2TP + FP + FN}.$$

Não se faz média dos índices dos tiles sobrepostos. Denominadoreus nulos produzem valor ausente. Áreas em km^2 são informadas apenas quando a área constante do pixel em m^2 é fornecida; sem esse dado, retornam-se contagens.

3.2. Eleição de líder em anel

A eleição implementa as regras de Chang-Roberts [Chang e Roberts 1979]: um nó inicia enviando seu ID ao sucessor. Ao receber um candidato de maior valor, o mesmo é propagado, enquanto caso um candidato menor é recebido, se faz uma substituição pelo ID do nó atual, se ainda não participa, ou descartado se já participa. Ao receber o próprio ID, o nó se elege e propaga `Coordinator`. Em um anel estável, o pior caso usa $O(n^2)$ mensagens de eleição, mais $O(n)$ para anunciar o vencedor.

A aplicação acrescenta rodadas ordenadas por contador persistido e ID iniciador, rejeitando mensagens antigas. Antes de iniciar uma rodada, consulta os contadores dos pares para permitir o retorno de um nó atrasado. Heartbeats utilizam tempo monotônico local. A ausência de resposta fora de um intervalo determinado gera suspeita e dispara uma nova eleição. O timeout não constitui prova de falha.

Nesta etapa, os jobs ficam guardados apenas na memória do coordenador. Se ele falhar, a eleição escolhe um novo líder, que já consegue receber trabalhos novos, mas não tem o histórico dos anteriores. Por isso, um job que estava em andamento precisa ser reenviado pelo cliente. Já as falhas de comunicação com os workers são tratadas pelo próprio líder, que reenvia o tile a outro worker.

4. Testes da primeira entrega

Foram realizados testes locais com processos Python independentes, comunicando-se por gRPC, abrangendo a divisão das imagens em tiles e sua reconstrução, o tratamento de sobreposições e pixels inválidos, o cálculo de métricas, a validação do formato NPY e a leitura de arquivos NetCDF. Também foram verificadas a execução com dois e três nós, a idempotência das submissões ao líder atual e a eleição de líder. Para validar a inferência foi utilizado os patches de teste referente ao split 1 da base DL4GAM [Diaconu et al. 2024], como os patches são divisões de 256×256 pixels da imagem original da geleira parametrizamos para subdividir em tiles de 128×128. A inferência ocorreu conforme o esperado com a fragmentação e reconstrução correta do patch original, o mesmo quando simulado a queda de um nó. Os códigos referentes ao presente trabalho se encontram em: https://github.com/Gabriel3637/Computacao$_{D}istribu - da_{P}rojeto_{E}tapa_{1}.git$

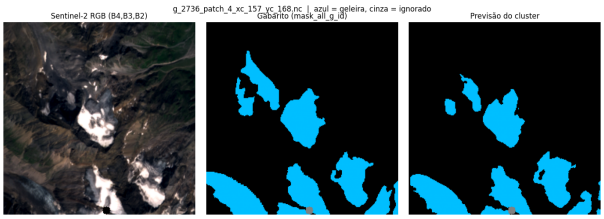


Figura 2. Patch da geleira, gabarito e segmentação do modelo

5. Desafios e responsabilidades

Foram tratados a dupla contagem nas sobreposições, a divergência de versões do modelo e as mensagens antigas de eleição. As soluções foram costurar antes de calcular métricas, identificar o modelo por hash e ordenar as rodadas. Caminhos longos de arquivos no Windows também receberam tratamento explícito. Timeout gera suspeita de falha; o protocolo não impede líderes diferentes em partições da rede.

Em cada dupla, teoria e implementação são compartilhadas. Bruno Menezes Rodrigues Oliveira Vaz e João Pedro Torres respondem por tiles, costura e métricas globais; Gabriel Valedo Batista Silva e Davi Cândido de Almeida, pelo contrato Protobuf, comunicação gRPC e integração U-Net; e Vitor Leite Setragni e Welbert Junio Afonso de Almeida, pela eleição em anel, heartbeat e testes de troca de coordenador.

Referências

Chang, E.; Roberts, R. (1979). An improved algorithm for decentralized extrema-finding in circular configurations of processes. *CACM*, 22(5), 281–283. doi:10.1145/359104.359108.

gRPC (2026). Basics tutorial: Python. Documentação oficial. Acesso em 25 out. 2026.

gRPC (2026). Deadlines. Documentação oficial. Acesso em 25 out. 2026.

Ronneberger, O.; Fischer, P.; Brox, T. (2015). U-Net: Convolutional Networks for Biomedical Image Segmentation. *MICCAI*, 234–241. arXiv:1505.04597.

Diaconu, C. A.; Zekollari, H.; Bamber, J. L. (2024). DL4GAM: a multi-modal deep learning-based framework for glacier area monitoring, trained and validated on the European Alps. *ESS Open Archive*, 2024(1230). doi:10.22541/essoar.173557607.70204641/v1.